جامعة هواري بومدين للعلوم والتكنولوجيا
USTHB

FACULTÉ D'INFORMATIQUE

CHAPTER 3- PART 2

QUERYING RELATIONAL DATABASE

SQL

Zakia Challal

zakiachallal@gmail.com

Computer Science Engineering. 2nd year. 2024-2025

Content

01 Analogy with Relational Algebra:

- Projection
- Selection
- Union, Intersection, Difference
- Cartesian Product
- Join
- Division

02 Other queries:

- Aggregation function (Sum, Avg, Max, Min,…)
- Group by
- Having
- Sub Queries
- Views

Introduction

- In last chapters, we studied how to query a relational database through relational algebra operators.

- In this chapter, we study how to practically query a database on a DBMS using SQL.

  Note: In this course, we present Oracle SQL (which can differ slightly from other DBMS).

SQL

Course Example

Student

SID	Name	Major
S1	Alice	Computer Science
S2	Bob	Mathematics
S3	Carol	Computer Science

Course

CID	Title	Credits
C1	Databases	3
C2	AI	4

Enrolled

SID	CID	Grade
S1	C1	A
S1	C2	B
S2	C2	A

Projection

Definition:

The projection operation is used to retrieve specific columns from a relation (removes duplicates).

Syntax:

```
SELECT columns_names
FROM Table_name ;
```

Note: Use * to retrieve all columns.

We can use alias for table_name. Ex: select **t**.column_name from table_name **t**;

Examples:

Get all student names.

SELECT name FROM Student;

Get all student's columns for all student.

SELECT * FROM Student;

Selection (or Restriction)

Definition: The selection operation is used to retrieve rows from a relation that satisfies a given condition.

Syntax:

```
SELECT columns_names
FROM Table_name
WHERE condition;
```

- Note: in condition use operators =, >, >=, <, <=, <> , BETWEEN, LIKE, IN, EXISTS combined with AND, OR, NOT.
- SQL do not remove duplicate. Use **DISTINCT** before columns names to have distinct values
- Use **ORDER BY** column_name at the end to order results (use asc, desc).

Example: Get students majoring in Computer Science.

```
SELECT * FROM student WHERE major='Computer Science';
```

Union

Definition: The union operation combines the results of two relations, eliminating duplicates. It requires both relations to have the same number of attributes with matching domains.

Syntax:

```
SELECT columns_names FROM Table_name      WHERE condition
UNION
SELECT columns_names FROM Table_name      WHERE condition;
```

Example:

Get students name majoring in Computer science or Mathematics

```
SELECT name FROM Students WHERE major = 'CS'
UNION
SELECT name FROM Students WHERE major = 'Math';
```

Intersection

Definition: The intersection operation returns the rows that are common to both relations. It also requires the same structure for both relations.

Syntax:

```
SELECT columns_names FROM Table_name        WHERE condition
INTERSECT
SELECT columns_names FROM Table_name        WHERE condition;
```

Example:

```
SELECT student_id FROM Enrolled WHERE Course_id='C1'
INTERSECT
SELECT student_id FROM Enrolled WHERE Course_id='C2';
```

Difference (-)

Definition: Gives tuples in one relation but **not** the other.

Syntax:

```
SELECT columns_names FROM Table_name  WHERE condition

MINUS

SELECT columns_names FROM Table_name  WHERE condition;
```

Example:

Students not enrolled in any course.

SELECT sid FROM student MINUS SELECT sid FROM Enrolled;

Other solution:

SELECT name FROM Student WHERE sid NOT IN (SELECT sid FROM Enrollments);

Cartesian Product

Definition:

Combines every row from one relation with every row from another.

Syntax:

```
SELECT columns_names
FROM Table_name1, Table_name2  ;
```

Example:

Combine all students with all courses

SELECT * FROM student, courses;

SID	Name	Major	CID	Title	Credits
S1	Alice	Computer Science	C1	Databases	3
S1	Alice	Computer Science	C2	AI	4
S2	Bob	Mathematics	C1	Databases	3
S2	Bob	Mathematics	C2	AI	4
S3	Carol	Computer Science	C1	Databases	3
S3	Carol	Computer Science	C2	AI	4

Natural Join

Definition: A natural join automatically joins two relations by matching columns with the same names and combines the rows where these columns have equal values.

Syntax:

```
SELECT columns_names
FROM Table_name1, Table_name2
WHERE Table_name1. column_name = Table_name2. column_name;
```

Example: Get students with their enrolled courses.

```
SELECT *
FROM Student, Enrolled
WHERE Student.Sid=Enrolled.Sid;
```

Theta Join

Definition: A theta join combines rows from two relations based on a condition (theta) that can be any comparison operator (e.g., =, <, >, <=, >=, ≠).

Syntax:

```
SELECT columns_names
FROM Table_name1, Table_name2
WHERE Table_name1. column_name1 Op Table_name2. column_name2;
```

Op can be any comparison operator (e.g., =, <, >, <=, >=, ≠)

Example:

Division

Definition: The division operation is used to find tuples in one relation that are related to all tuples in another relation. It is typically used for **"all"** queries.

Syntax:

No special operator for the Division.

Think about it.

Example:

Students who took **all** courses.

Aggregation Functions

Aggregation functions in SQL are used to perform **calculations across multiple rows of a table.**

Common SQL Aggregation Functions are:

COUNT(), SUM(), AVG(), MIN(), MAX()

Syntax:

```
SELECT Function_Name(Column_name)
FROM tabl_name;
```

Example:

How many students are their? `SELECT COUNT(SID) FROM Student;`

Which is the maximum course credit? `SELECT MAX(credit) from Course;`

Group by

GROUP BY groups rows that have the same values in specified columns into summary rows

Syntax:

```
SELECT column1, AGG_FUNC(column2)
FROM table
GROUP BY column1;
```

Note: the column in the goup by must appear in the Select.

Example: Number of student by course?

```
SELECT CID, COUNT(SID)
FROM Enrolled
Group by CID;
```

Total credits taken by each student ?

```
select s.student_id, sum (credits) from students s , courses c, enrollments e
where s.student_id=e.student_id and e.course_id=c.course_id group by
s.student_id;
```

Having

Having is often used with **GROUP BY** to filter grouped data based on aggregate values.

- WHERE filters individual rows before grouping.
- HAVING filters groups of rows after GROUP BY has been applied.

Syntax:

```
SELECT column1, AGG_FUNC(column2)
FROM table
GROUP BY column1
HAVING AGG_FUNC(column2) condition;
```

Example: Students with total credits > 6

```
SELECT s.name, SUM(c.credits)
FROM Students s, Courses c, Enrollments e
WHERE s.student_id = e.student_id AND e.course_id = c.course_id
GROUP BY s.name
HAVING SUM(c.credits) > 6;
```

General Syntax of a SELECT Statement

```
SELECT [DISTINCT] column1, column2, ...
FROM table_name
[WHERE condition]
[GROUP BY column1, column2, ...]
[HAVING condition]
[ORDER BY column1 [ASC|DESC], ...]
```

The END